



UNIVERSIDAD POLITECNICA SALESIANA

Materia: Programación Orientada a Objetos

Examen Practico: Programación del juego Buscaminas

Integrantes

- Geovanny Steven Sánchez Maza
- Cristian Eduardo García Torres
- Luis Fernando Fuelagan Pujota
- Steven Javier Estupiñan Guaman

Curso: Grupo 1

Fecha:05/06/2026

Contenido

Resumen	3
Introduccion	3
Objetivos	3
Objetivo General	3
Objetivos específicos	4
Fundamentación Teórica	4
Arquitectura del sistema	5
Estructura del proyecto y paquetes	5
Diagrama de clases UML.....	6
Implementación de componentes críticos	6
Pruebas Unitarias y validación de calidad	8
Conclusiones	8
Referencias	9
Anexos	9

Resumen

El siguiente proyecto muestra el diseño, desarrollo e implementación de una aplicación interactiva del videojuego “Buscaminas” programada en el lenguaje Java. El objetivo es en aplicar de manera profesional los pilares de la Programacion Orientada a Objetos (POO) y el patron arquitectónico Modelo-Vista-Controlador (MVC) para lograr un sistema mantenible y escalable. La aplicación incorpora características como persistencia de datos mediante serialización binaria para el guardado y carga de partidas y un control de flujo basado en excepciones verificadas, de igual manera, la confiabilidad de reglas de negocio se baso en el enfoque de Desarrollo de Pruebas Unitarias (TDD) con el framework Junit, garantizando así el desarrollo y aplicación de un software robusto.

Introduccion

En el desarrollo de software, la implementación de patrones de desarrollo y arquitectura es indispensable construir aplicaciones que puedan ser mantenidas y evolucionadas evitando inutilizar el programa codificado. La Programación Orientada a Objetos proporciona abstracciones naturales del mundo real mediante clases y objetos, encapsulando datos y comportamientos, además para evitar que se mezclen se vuelve necesario aplicar patrones de modelado como el patron Modelo-Vista-Controlador (MVC)

Objetivos

Objetivo General

Diseñar e implementar un software interactivo del videojuego Buscaminas en el lenguaje JAVA, aplicando los conceptos y principios de la programación Orientada a Objetos, el patrón de diseño MVC, el manejo formal de excepciones y pruebas unitarias automatizadas para asegurar una arquitectura limpia y robusta

Objetivos específicos

- Separar las responsabilidades del sistema aislando la lógica del negocio (Modelo), la interfaz de texto de consola (Vista) y la orquestación del flujo de datos (Controlador)
- Garantizar la estabilidad de la aplicación mediante la creación y captura de excepciones que eviten jugadas invalidas o repetidas.
- Implementar un componente de persistencia de datos para guardar y reanudar la partida a través de un medio secundario local empleando flujos binarios
- Validar el comportamiento algorítmico del tablero y sus componentes mediante suites de pruebas JUnit, promoviendo la calidad de software desde sus etapas de desarrollo

Fundamentación Teórica

- **Programación Orientada a Objetos (POO):** Paradigma basado en el concepto de "objetos" que contienen datos (atributos) y código (métodos). Se fundamenta en cuatro pilares: Encapsulamiento (ocultamiento de datos), Abstracción (definición de características esenciales), Herencia (reutilización de código) y Polimorfismo (múltiples formas de comportamiento).
- **Patrón Arquitectónico Modelo-Vista-Controlador (MVC):** Patrón que divide una aplicación en tres componentes interconectados. El *Modelo* gestiona directamente los datos, reglas y lógica; la *Vista* renderiza la representación de los datos para el usuario; y el *Controlador* acepta entradas del usuario, convirtiéndolas en comandos para el modelo o la vista.
- **Persistencia y Serialización Binaria:** Capacidad de un sistema de software de trascender el tiempo de ejecución de un proceso, guardando su estado en un almacenamiento permanente. En Java, la interfaz *Serializable* permite transformar un grafo de objetos en un flujo de bytes para su almacenamiento en disco.

- **Excepciones Verificadas (Checked Exceptions):** Eventos anómalos que ocurren durante la ejecución de un programa y que derivan directamente de la clase Exception. El compilador de Java obliga explícitamente al desarrollador a manejarlas (mediante bloques try-catch) o declararlas (usando throws), forzando un diseño de software altamente predictivo.
- **Desarrollo Guiado por Pruebas (TDD) y JUnit:** Práctica de ingeniería de software donde los casos de prueba se escriben antes del código que se va a probar. JUnit es el framework estándar en el ecosistema Java para ejecutar pruebas unitarias de forma repetible, asegurando que los refactorizajes no alteren el comportamiento correcto del sistema.

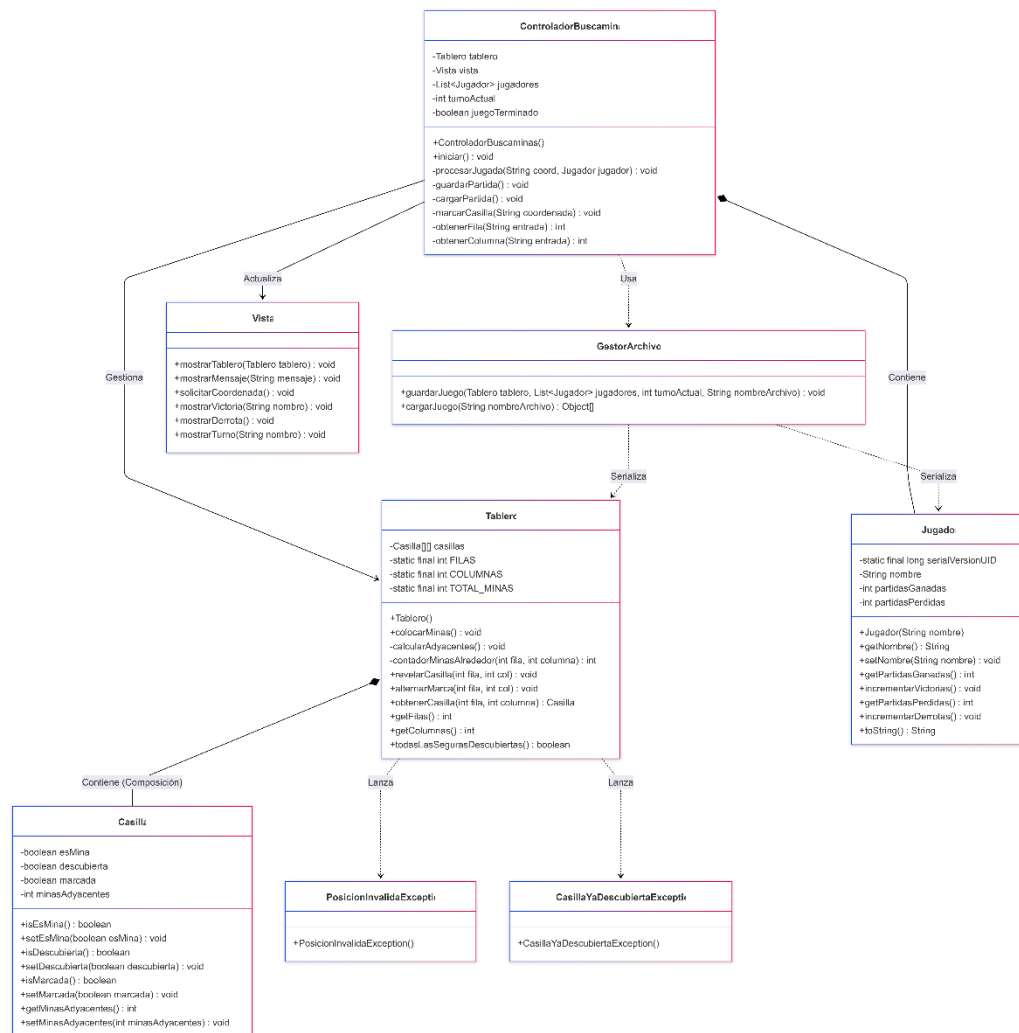
Arquitectura del sistema

Estructura del proyecto y paquetes

La solución de software está organizada de manera modular en paquetes específicos dentro del directorio raíz de fuentes (src), garantizando la separación física y lógica de las capas del patrón arquitectónico:

- **principal:** Contiene el punto de entrada de la aplicación (Main.java).
- **modelo:** Aloja las entidades de negocio y estado del dominio (Casilla.java, Tablero.java, Jugador.java).
- **vista:** Controla la representación textual e interfaz de usuario en consola (Vista.java).
- **controlador:** Orquesta el juego y gestiona los archivos físicos en disco (ControladorBuscaminas.java, GestorArchivos.java).
- **excepciones:** Define los errores de negocio específicos (PosicionInvalidaException.java, CasillaYaDescubiertaException.java).
- **test/modelo:** Suite paralela que contiene los entornos de pruebas unitarias (CasillaTest.java, TableroTest.java).

Diagrama de clases UML



Implementación de componentes críticos

1. La clase **Tablero** inicializa una matriz bidimensional estática de objetos **Casilla**. Al descubrir una casilla sin minas adyacentes ($\text{minasAdyacentes} == 0$), el método `revelarCasilla` propaga el desvelado de forma recursiva hacia sus ocho vecinos directos. Esta propagación lógica se controla mediante condiciones de frontera y el estado `isDescubierta()`, evitando bucles infinitos:

```

// Verifica que la posicion exista
if (fila < 0 || fila >= FILAS || col < 0 || col >= COLUMNAS) {
    throw new PosicionInvalidaException();
}
Casilla casilla = casillas[fila][col];

// Verifica que no este ya descubierta
if (casilla.isDescubierta()) {
    throw new CasillaYaDescubiertaException();
}
// No revela casillas marcadas
if (casilla.isMarcada()) {
    return;
}
// Descubre la casilla
casilla.setDescubierta(descubierta: true);

// Si no tiene minas alrededor y no es mina, revela las casillas vecinas tambien
if (!casilla.isEsMina() && casilla.getMinasAdyacentes() == 0) {
    for (int i = fila - 1; i <= fila + 1; i++) {
        for (int j = col - 1; j <= col + 1; j++) {
            if (i >= 0 && i < FILAS && j >= 0 && j < COLUMNAS) {
                if (!casillas[i][j].isDescubierta()) {
                    revelarCasilla(i, j);
                }
            }
        }
    }
}
}

```

2. El controlador admite partidas simultáneas para un arreglo dinámico de objetos Jugador. La alternancia del jugador activo se resuelve mediante el operador residuo o módulo (%), lo que asegura un ciclo cerrado infinito (del jugador 1 al 4, y retornando de nuevo al 1 al superar los límites del índice de la lista):

```

turnoActual = (turnoActual + 1) % jugadores.size();

```

3. La clase GestorArchivos centraliza el almacenamiento persistente. Al utilizar el bloque *try-with-resources*, el sistema garantiza el cierre seguro de los flujos de

```

public class GestorArchivos {
    public void guardarJuego(Tablero tablero, List<Jugador> jugadores, int turnoActual, String nombreArchivo) throws IOException {
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(nombreArchivo))) {
            oos.writeObject(tablero);
            oos.writeObject(jugadores);
            oos.writeInt(turnoActual);
        }
    }
    @SuppressWarnings("unchecked")
    public Object[] cargarJuego(String nombreArchivo) throws IOException, ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(nombreArchivo))) {
            return new Object[] { (Tablero) ois.readObject(), (List<Jugador>) ois.readObject(), ois.readInt() };
        }
    }
}

```

comunicación con el sistema operativo ante cualquier fallo en operaciones de Entrada/Salida (I/O):

Pruebas Unitarias y validación de calidad

La verificación de la lógica matemática del modelo se sustentó bajo pruebas automáticas con JUnit en el directorio prueba/modelo. La metodología TDD aplicada permitió comprobar de manera aislada y controlada los siguientes escenarios operacionales críticos:

- **Validación de Fronteras e Integridad:** Pruebas enfocadas en verificar que el método `revelarCasilla` dispare de forma controlada una `PosicionInvalidaException` si los parámetros numéricos de entrada de la matriz exceden el rango válido (menor a 0 o mayor a 9 en un tablero de 10x10).
- **Validación de Idempotencia en Jugadas:** Verificación de que una casilla ya descubierta mantenga su estado invariante y lance una `CasillaYaDescubiertaException` al intentar interactuar con ella consecutivamente, protegiendo al flujo principal de acciones redundantes.
- **Consistencia Estocástica:** Inspección automatizada para corroborar que el algoritmo de inicialización del tablero distribuya de manera exacta la cantidad paramétrica configurada de minas (10 elementos), ni más ni menos, resguardando la equidad matemática del juego.

Conclusiones

- La separación arquitectónica mediante el patrón MVC demostró ser sumamente eficiente; la capa del modelo opera de forma pura e independiente de los flujos de entrada/salida de la consola
- El manejo de excepciones propio elevó la robustez de la aplicación, convirtiendo errores de interacción del operador en eventos predecibles y controlados en lugar de fallos críticos que provoquen la caída súbita del entorno de ejecución (JVM).

Referencias

- Universidad Politécnica Salesiana. (2026). *Programación Orientada a Objetos*. Texto guía institucional / Documento PDF de la asignatura. Carrera de Ingeniería de Software.

Anexos

Enlace de GitHub: https://github.com/crisedugarciat593-stack/Examen_Practico_POO.git